

CalSoft, Rating, Comparison with the Sibling Calibration Modules, and What It Needs

Repository: cirqen-labs | Branch: improvement-plan | Reviewed: 17 September 2026

Re-scored 18 September 2026 after implementation, see section 2a. Companion to A1_ENGINEERING_AUDIT.pdf (the formula-level audit)

1. How this rating was produced

Eleven dimensions, each weighted by how much it matters **for a module whose output is a signed conformity statement about clinical equipment**. That weighting is the whole point: a bug in a dashboard is an inconvenience, and a bug in a pass/fail verdict puts a device back on a ward. Decision-rule integrity and output fidelity therefore carry more weight than, say, shared service adoption.

Each module is scored 1-5 per dimension against observable evidence in the tree, call sites, test assertions, formula derivations, not impressions. Three comparators were scored alongside CalSoft:

- calSchedules, the scheduling half of calibration. The closest sibling: same domain, same data, same team, overlapping models.
- ppms, preventive maintenance. A parallel domain with the same shape (schedules, cycles, PDF reports) but no measurement mathematics.
- sync + updates, the distribution and integrity layer. Included because it is where the platform's security engineering actually lives.

What this rating does and does not cover

The four modules scored here all live in the cirqen-labs repository. The platform extends beyond it: ~/Desktop/hq_server is a **separate** git repository of 47 modules and 25,542 lines that acts as the HQ sync and certificate authority, it allocates certificate numbers and flips sessions to approved. (Confusingly, cirqen-labs/hq_server/ is a different codebase again, the software update server.)

HQ is **not scored** here. Its sync, mirror and soft-delete layers have not been audited, and inventing a number for them would be worse than leaving the gap visible. Two facts about it are relevant to the scores below and are used in Section 4.2 and section 4.3:

- HQ carries **41 tests** across 25,542 lines, aimed at what can corrupt data: certificate numbering, certificate delivery, soft-delete cascade, last-write-wins upsert, null propagation, API keys.
- HQ already runs **per-device API keys** with hashing, issuance and revocation

and the platform already runs **Ed25519** signing for update packages.

2. The scorecard (baseline, 17 September)

Dimension	Weight	CalSoft	calSchedules	ppms	sync+updates
Core formula correctness	15	4	5	3	4
Decision-rule integrity	15	1	4	3	4

Dimension	Weight	CalSoft	calSchedules	ppms	sync+updates
Output fidelity / auditability	12	2	4	3	3
Test coverage of core logic	12	1	4	1	4
Single source of truth	10	2	5	2	3
Failure visibility	10	2	4	3	3
Security of issued artefact	8	1	3	3	5
Transaction / data integrity	6	4	4	2	4
Documentation	5	4	4	1	3
Pure/IO separation (testability)	4	4	5	2	4
Shared-service adoption	3	3	4	4	4

Module	Weighted score
calSchedules	84 / 100
sync + updates	74 / 100
ppms	50 / 100
CalSoft, baseline	46 / 100
CalSoft, after implementation (section 2a)	81 / 100

At baseline CalSoft rated lowest of the four, and it is the module carrying the highest consequence. That combination, not the score itself, was the finding. After implementation it rates second of the four, behind calSchedules; section 2a records the re-score and what is left.

The score is not a verdict on the engineering skill in the module. The mathematics is competent and in places better than its siblings. CalSoft scores 46 because the competent parts are not wired to the parts that print the verdict, and because nothing tests them.

2a. Re-score after implementation

The 4.6 above was the state on 17 September. Track A phases 1, 2, 3 and most of 5 have since been implemented, along with Track B item B1. Re-scored on the same eleven dimensions and the same weights:

Dimension	Weight	Was	Now	Target
Core formula correctness	15	4	4	5
Decision-rule integrity	15	1	4	5
Output fidelity / auditability	12	2	5	5
Test coverage of core logic	12	1	5	5
Single source of truth	10	2	4	5

Dimension	Weight	Was	Now	Target
Failure visibility	10	2	3	4
Security of issued artefact	8	1	2	5
Transaction / data integrity	6	4	4	5
Documentation	5	4	5	5
Pure/IO separation	4	4	5	5
Shared-service adoption	3	3	3	4

	Score
Baseline, 17 September	4.6 / 10
Now	8.1 / 10
Plan target	9.7 / 10

What moved most. Test coverage 1 -> 5: the module went from no assertion on any computed value to **181 tests**, of which **95 run with no database at all**, every formula pinned to a hand-computed vector, in 0.007 seconds. Decision-rule integrity 1 -> 4: the certificate reads strict conformity instead of a 40% failure-rate threshold, and guard banding now produces PASS / FAIL / INDETERMINATE with the rule printed. Output fidelity 2 -> 5: the budget reconciles on the page, n and the applied coverage factor are printed, drift is graded against tolerance, and linearity reports slope, intercept and residual instead of an unannotated chart.

What did not move, and why.

- **Core formula correctness stays at 4.** Effective degrees of freedom is implemented and k is now derived rather than assumed, which is real progress. But 5 requires sensitivity coefficients, and until every component can carry one the budget still cannot represent an influence quantity that acts indirectly. That is P5.2 and it needs a schema change.
- **Security of the issued artefact moves only 1 -> 2.** B1 removed local certificate-number allocation, so numbers are no longer invented or duplicated, a genuine improvement. But the QR verifier still returns valid for any input and nothing is signed, so the artefact is not tamper-evident. This is the largest remaining gap by weight: **+0.5 of the missing 1.6**.
- **Failure visibility 2 -> 3.** The two features that reported confident wrong answers now report real ones and log when they cannot. 73 broad exception handlers remain in the module.

Where the remaining 1.6 points sit, largest first: security of the issued artefact (0.5), core formula correctness (0.3), decision-rule integrity (0.3), single source of truth (0.2), failure visibility (0.2), then transactions and shared services (0.1 each). Only the first needs the HQ repository; the rest are local work, and two of them are blocked on decisions rather than effort, see 04_PLAN_TO_9_OF_10.pdf section 12.

3. Where CalSoft genuinely leads

Stated first, because the rest of this document is critical and the balance matters.

Mathematical depth. No sibling module attempts anything comparable. caLScheduLes does date arithmetic; ppms does interval counting. CalSoft implements a GUM-shaped uncertainty budget, Bessel-corrected s, $u_A =$

$s/\sqrt{n}$, a rectangular resolution component, a reference component reduced from expanded to standard, RSS combination, and each of those five formulas is individually correct. The Iglewicz-Hoaglin modified Z-score uses the right constants. The drift regression, R^2 included, is correctly derived.

Transaction discipline, best in the repository. 12 uses of `transaction.atomic / select_for_update`, against 8 in `calSchedules` and 2 in `ppms`. `generate_certificate_number` mints inside a locked transaction, which is the correct way to serialise concurrent issuance and is more careful than anything in the sibling modules. One qualification, added after reviewing the HQ server: in normal online operation HQ is the allocator, under its own advisory lock. Both implementations are careful; the problem is that there are two of them (section 4.3).

Documentation, best in the repository by an order of magnitude.

`CALIBRATION_MODULE_TECHNICAL_DOCUMENTATION.md` runs 6,008 lines against `SCHEDULING_NOTES.md` at 90 and `sync/ARCHITECTURE.md` at 56. It derives every equation from first principles, cites file and line throughout, and carries a defect register (D-01 ... D-24) with severities. It documents things this audit had to rediscover independently, the unbound `calculate_linearity` call, the non-existent `TrendAnalysis.calculate_linear_regression`, the even-n quasi-median, the guard-band `u_c-versus-U` ambiguity, and a client/server divergence in the error sign that the formula audit missed. Section 5.9.2 even works a TUR evaluation by hand.

The math layer is already pure. `CalibrationCalculator` imports only `math` and `decimal`, no Django, no ORM (`utils.py:101-103`). It can be unit-tested with `SimpleTestCase` **today, with zero refactoring**. This is the single most important fact in this document, and section 6 builds on it.

`backfill_calibration_stats` is the best repair command in the tree: it documents the bug it fixes, recomputes dependents, backfills the history table and implements `--dry-run` through a sentinel rollback.

4. Where CalSoft trails, and the sibling that already solved it

4.1 Single source of truth, `calSchedules` solved this exact disease

CalSoft carries **four parallel uncertainty implementations** (`calculate_uncertainties`, `CalibrationCalculator`, the inline arithmetic in `models.py`, and the inline arithmetic in `api_calculate_uncertainty`), **two drift engines** on two different data sources, and **two linearity implementations**, neither of which reaches a certificate.

`calSchedules/grouping.py` had the same disease and cured it. Its docstring is worth quoting, because it is a template:

```
Historically the same concepts were re-implemented across tasks.py,
instant_reconciliation.py, locker.py and reconciliation.py ...
Those copies had DIVERGED (e.g. some filtered on the schedule's
active_status while others filtered on the equipment's), so they are NOT
freely interchangeable. This module centralises the logic while preserving
every caller's behaviour ...
```

```
Design rule: everything above group_members_qs is pure (no ORM, no
side effects) and unit-tested with SimpleTestCase.
```

That design rule is exactly what CalSoft's math layer needs, and CalSoft is

closer to satisfying it already than `calSchedules` was, because `CalibrationCalculator` is pure from birth. What is missing is the decision to make it the only implementation, and the tests that pin it.

`calSchedules` scores 5 here; CalSoft scores 2.

4.2 Test coverage of core logic, the defining gap

Module	Tests	LOC	Per kloc	SimpleTestCase (no-DB)
workshop	16	790	20.3	0
Inventory	70	8,576	8.2	6
users	30	4,836	6.2	0
calSchedules	39	8,869	4.4	8
core	,	,	,	20
CalSoft (baseline)	35	12,326	2.8	0
CalSoft (now)	181	14,738	12.3	95
sync	48	16,982	2.8	0
ppms	5	5,698	0.9	0
reporthub / machineReports / parts_tools	0	9,370	0.0	0

CalSoft's 2.8 per kloc is mid-pack, and three modules have no tests at all, so raw density is not the problem. The problem is *what* is tested. Across tests.py, test_calibration_flow.py and test_imports.py, 564 lines, 35 tests, **there is not one assertion on a mean, a standard deviation, an uncertainty component, a tolerance decision or a drift rate.** tests.py:141 tests linearity by mocking out the very regression call that is broken, so the suite passes green while the feature has never worked.

calSchedules shows the alternative: tests/test_grouping.py asserts pure logic against hand-computed values with SimpleTestCase, no database, in milliseconds. core has 20 such tests.

The HQ server sharpens the point further. It has a dedicated tests/test_cert_numbering.py asserting exactly how a certificate number is allocated, sequential, gap reuse, custom prefixes, empty table. So the platform

tests the allocation of the number on the certificate and tests none of the measurement mathematics printed beneath it. More care has gone into the identifier than into the measurement it identifies.

Every finding in A1_ENGINEERING_AUDIT.pdf section 7 would have been caught by one test per formula.

4.3 Security of the issued artefact, the platform already has the answer

CalSoft's verify_certificate_qr (verification.py:195) returns 'valid': True **unconditionally**, with the comment "This would be determined by database lookup". generate_verification_url base64-encodes cert:session_id:timestamp with no HMAC, so the token is forgeable by anyone who can read one certificate. Note 4 on every certificate reads "Certificate authenticity can be verified by scanning the QR code." PDFConfig records 'digital_signature': False.

Meanwhile the same repository already runs **Ed25519 signing end to end**:

- hq_server/build_package.py:53, _sign_bytes() signs manifest bytes with a private key held only in the server environment.
- updates/updater.py:81-97, the client loads the embedded public key and refuses anything that fails verification.
- cryptography is already a dependency; key generation is already a --genkeys flag.

The recommendation is therefore not "adopt cryptography". It is "reuse the pattern already running in the platform." sync+updates score 5 on this dimension; CalSoft scores 1. That is the widest gap on the scorecard.

One correction to where that work lands. Certificate signing is **not** a CalSoft change: certificate numbers are allocated by the separate HQ server and a signature is only as meaningful as the identity it binds, so signing belongs with the component that has authority to allocate. That also brings a second issue into the same piece of work, two different numbering algorithms run on two different databases with locks that cannot see each other, one reusing gaps and one strictly incrementing. 04_PLAN_TO_9_OF_10.pdf section 6 covers both together, and is no longer the cheapest phase in the plan as a result.

4.4 Decision-rule integrity, no sibling has this failure mode

session.overall_pass (strict conformity: every point must pass) is computed at view_modules/calibration.py:232-280 and then **never read by any module** in pdf_generators/. The printed verdict, the header banner, certificate note 5 and the QR STATUS field all derive from a second, unrelated rule at certificate.py:95:

```
is_failed_report = (failed_readings / total_readings) >= 0.40
```

A device failing up to 39% of its test points is certified **PASSED** while the database records it as failed. No sibling module has anything comparable because no sibling module issues a conformity statement. This is CalSoft's score of 1, and it is the reason the module rates below ppms despite being far more sophisticated.

Compounding it: measurement uncertainty is computed, printed, and then excluded from the decision (models.py:680 is a bare `|error| <= tolerance`). MetrologyUtils.calculate_guard_banding and calculate_measurement_capability implement the missing rules and have no call sites.

4.5 Failure visibility

Two features report confident wrong answers rather than failing: sessions.py:126 reads five keys DriftAnalyzer.analyze_drift never returns so the session page prints "No significant drift detected" for **every** device regardless of data; sessions.py:228 calls a TrendAnalysis method that does not exist, and a bare except Exception degrades every parameter to "Unable to calculate linearity".

CalSoft's 77 except Exception across 12.3k lines (6.2 per kloc) is not an outlier, ppms runs 9.1 and Inventory 6.9. The difference is consequence: in Inventory a swallowed exception loses an import row, in CalSoft it silently replaces a drift verdict with a reassuring sentence.

4.6 Output fidelity

The printed uncertainty budget omits the reference component column, so Combined $!= \sqrt{\text{TypeA}^2 + \text{TypeB}^2}$ on the page and the arithmetic does not reconcile for an assessor. All six uncertainty columns print at :.4f against values stored at 6 dp, so fine-resolution parameters print a column of zeros. Note 3 asserts k=2 unconditionally while the adjacent column renders the per-parameter coverage_factor. The Error column carries the opposite sign to the metrological convention, and, per the module's own documentation (line 413), the **client-side preview uses the opposite sign again**, so the technician's screen and the certificate disagree.

5. The documentation finding, which cuts both ways

CalSoft's technical documentation is the best artefact in this repository. It is also **stale on the single item it labels most important**, and that is worth its own section because it changes how the document should be used.

Section 6.10 is titled *"Divergence: the web submission path does not compute error at all"* and opens: *"This is the most consequential finding in the audit and is stated in full here."* It then states that `CalibrationReading.calculate_statistics()` **"has no caller ... The model method is dead code"**, and that what runs on submission is a helper called `_calculate_stats`.

Neither claim holds in the current tree:

Doc claim (v1.4.8)	Current tree
<code>calculate_statistics()</code> has no caller	Called at <code>view_modules/calibration.py:276</code> and <code>backfill_calibration_stats.py:83</code>
<code>_calculate_stats</code> runs on submission	Does not exist; referenced only in <code>backfill_calibration_stats.py</code> as "fixed"

The bug was fixed and the repair command written; the documentation was not updated. The doc's last commit is `a208bcf` version 1.4.8.

The consequence is specific and costly: a new engineer or an ISO/IEC 17025 assessor reading the handbook today would hunt for a defect that no longer exists, and, worse, would read a **"dead code"** label on the method that is now the live calculation path for every certificate the hospital issues.

This is why documentation scores 4 rather than 5. The remedy is cheap: date the document, add a "verified against commit" line, and reconcile the defect register against the tree. The register's own D-14 ... D-18 entries remain accurate and should simply be carried into the issue tracker.

6. What you need, adopt, adjust, have

Split by the kind of action required, because these are three different pieces of work with three different risk profiles.

6.1 ADOPT, patterns already proven elsewhere in this repository

Nothing here needs designing. Each item has a working reference implementation two directories away.

Adopt	From	Closes
Ed25519 sign/verify for certificates	<code>hq_server/build_package.py:53</code> , <code>updates/updater.py:81</code>	Unverifiable QR; <code>digital_signature: False</code>
Pure-logic module + <code>SimpleTestCase</code> suite	<code>calSchedules/grouping.py</code> , <code>tests/test_grouping.py</code>	Zero math tests; 4 parallel implementations
Documented single-source-of-truth module with a divergence log	<code>calSchedules/grouping.py</code> docstring	Duplicate drift/linearity/uncertainty engines
<code>users.control.get_user_role</code> everywhere	already used in <code>CalSoft/view_modules/pending_sessions.py</code>	<code>certificates.py</code> reads <code>user_profile.role</code> raw in 5 places
<code>core.branding</code> for all identity strings	<code>CalSoft/pdf_generators/header.py</code> already does	Three competing identities: class name says "Btwelve Hospital", <code>pdf_config.py</code> says "KNH CALIBRATION LABORATORY", QR hardcodes "BTWELVE_NATIONAL_HOSPITAL"

Note on scope: CalSoft correctly does **not** use `core.scoping`. That is a documented design decision, not a gap, `core/scoping.py` states "Calibration schedules and sessions are deliberately not registered: the calibration centre serves every workshop, and those views are global by design." No change needed.

6.2 ADJUST, decisions to take in code that already exists

These need a judgement call recorded, then a small edit. The code is written; it is the decision that is missing.

1. **Pick one verdict.** Make the certificate read `session.overall_pass`. Demote the 40% rate to a maintenance-triage signal that never touches the conformity statement.
2. **Fix the error sign, or rename the column.** Either `mean - set_value` with a data migration, or keep the stored quantity and label it "Correction". Whichever is chosen, make the client-side preview agree, today it computes the opposite sign to the server.
3. **Resolve the guard-band ambiguity and wire it in.** The documentation (section 5.10) already lays out the `u_c`-versus-`U` question and warns the function "must not be wired in without a decision being taken and documented." Take the decision, then call `calculate_guard_banding` from the tolerance check.
4. **Make the printed budget reconcile.** Add the reference-uncertainty column.
5. **Replace unit-blind drift grades** with `|drift_per_year| / tolerance`

which is dimensionless and comparable across parameters. The current

0.1/0.5/1.0 constants grade mmHg, mV, °C and mL/min on one scale, and drive a printed recommendation to change a service interval.

1. **Enforce `num_readings` from below** and reconcile the four different minimums (2 in the model, 3 in the validator, 3 in `PDFConfig`, `num_readings` in the parameter).
2. **Print `n` and the actual `coverage_factor`** on the certificate; widen uncertainty formatting past : .4f.
3. **Restore `getcontext().prec` to a single value**, `utils.py:104` currently resets it from 28 to 12 process-wide as a side effect of a duplicated module header.

6.3 HAVE, capabilities that do not exist yet

Genuinely new work, in the order that buys the most defensibility per unit of effort.

1. **A test vector file.** One `SimpleTestCase` per formula, asserting against hand-computed values. `01_CALIBRATION_MATHEMATICS.pdf` section 15 is a ready first vector (5 readings -> `u_c = 0.488834`, `U = 0.977668`). This is the highest-leverage item in this document: `CalibrationCalculator` is already pure, so the cost is writing assertions, not refactoring.
2. **Effective degrees of freedom (Welch-Satterthwaite).** `calculate_degrees_of_freedom` is declared as this and its body is `return max(2, 10)`. With the 3-5 readings CalSoft actually operates on, a 95% interval needs `t` (2.78 at `nu = 4`), not 2.0, the app understates `U` by roughly 40% in its most common case.
3. **A real verification endpoint.** Look the certificate up, check the signature, return a truthful `valid`. Until then, remove note 4's claim from the certificate rather than print a promise the code does not keep.
4. **Sensitivity coefficients** in the budget model, so an influence quantity (temperature coefficient, hydrostatic head, lead resistance) can enter at all. Currently every `c_i` is implicitly 1.
5. **A stated decision rule on the certificate.** ISO/IEC 17025:2017 section 7.8.6 requires the conformity rule to be declared whenever a statement of conformity is issued, which is precisely what this certificate is.
6. **QA on the submission path.** `QualityAssurance.validate_readings` exists and runs only from an API endpoint; `_process_readings` calls nothing. Wire the outlier, variation and trend checks into submission.

7. **Documentation currency.** Date the handbook, record the commit it was verified against, reconcile the defect register, and move D-14 ... D-18 into the tracker.

7. Prioritised plan

Sequenced by consequence-per-effort, not by dimension order.

#	Work	Effort	Moves	Why first
1	Certificate reads overall_pass	Hours	Decision-rule 1->4	A non-conforming device currently gets a clean certificate
2	Test vectors for the 5 core formulas	1-2 days	Tests 1->4	Pins everything else; no refactoring needed
3	Error sign decided + client/server agree	Hours + migration	Output 2->3	Technician's screen and certificate disagree today
4	Reference column in printed budget	Hours	Output 3->4	The budget must reconcile for an assessor
5	Ed25519 certificate signing	1-2 days	Security 1->4	Copy an in-repo pattern; removes a false claim
6	Guard band wired, TUR warning	1 day	Decision-rule 4->5	Turns computed uncertainty into a used quantity
7	One math module, dead paths deleted	2-3 days	SSOT 2->4	Follow grouping.py; safe once step 2 exists
8	Drift/linearity call sites fixed + logged	1 day	Visibility 2->4	Stop printing reassurance in place of analysis
9	Tolerance-relative drift grades	1 day	Output 4->5	Makes the printed recommendation meaningful
10	Handbook reconciled and dated	Hours	Docs 4->5	Stops the register misleading the next reader

Steps 1-5 alone move the weighted score from **46 to roughly 70** and remove every finding that affects the correctness of an issued certificate.

8. The rating in one paragraph

CalSoft is the most mathematically ambitious module in this repository and the worst-governed. Its five core uncertainty formulas are correct, its transaction handling is the most careful in the tree, its documentation is the best by an order of magnitude, and its calculation engine is already pure enough to unit test without touching a line of it. None of that reaches the page, because the layer that prints the verdict consults a different rule from the layer that computed it, four parallel implementations of the same arithmetic disagree about which is authoritative, two analysis features return reassuring strings instead of failing, and not one test asserts on a single computed number. calSchedules scores 84 against CalSoft's 46 with less sophisticated logic, purely because it took the decisions CalSoft has not: one implementation, pure functions, no-database tests, divergences documented rather than duplicated. The path from 46 to 70 is five changes, none of them research, and the first one is a day's work.

Prepared from static analysis of the working tree at branch improvement-plan. No application code was modified. Scores are reproducible from the evidence cited; weights are a judgement about consequence and are stated in

section 2 so they can be argued with.

